2025115098_assigment4

1) Write a C program to implement a stack using queue data structure.

Source Code:

```
#include <stdio.h>

#define MAX 100

int q1[MAX], q2[MAX];
int front1 = 0, rear1 = 0;
int front2 = 0, rear2 = 0;

void push(int x)
{
    int i;

    q2[rear2++] = x;

    while (front1 < rear1)
    {
        q2[rear2++] = q1[front1++];
    }

    front1 = 0;
    rear1 = 0;

    for (i = front2; i < rear2; i++)
    {
        q1[rear1++] = q2[i];
    }

    front2 = 0;
    rear2 = 0;
}

int pop()
{
    return q1[front1++];
}

void display()
{
    int i;

    printf("\nStack elements: " );

    for (i = rear1 - 1; i >= front1; i--)
    {
        printf("%d ", q1[i]);
    }
}

int main()
{
    int n, i, x;

    printf("Enter number of elements: " );
    scanf("%d", &n);

    printf("Enter elements: " );

    for (i = 0; i < n; i++)
    {
        scanf("%d", &x);
        push(x);
    }

    display();

    printf("\nPopped element: %d", pop());

    return 0;
}
```

Output:

```
Enter number of elements: 5
```

```
Enter elements: 10 20 30 40 50

Stack elements: 50 40 30 20 10
Popped element: 50
```

2) Write a C program to implement a queue using stack data structure.

Source Code:

```
#include <stdio.h>

#define MAX 100

int stack1[MAX], stack2[MAX];
int top1 = -1, top2 = -1;

void enqueue(int x)
{
    stack1[++top1] = x;
}

int dequeue()
{
    if (top2 == -1)
    {
        while (top1 != -1)
        {
            stack2[++top2] = stack1[top1--];
        }
    }

    return stack2[top2--];
}

void display()
{
    int i;

    printf("\nQueue elements: " );

    for (i = top2; i >= 0; i--)
    {
        printf("%d ", stack2[i]);
    }

    for (i = 0; i <= top1; i++)
    {
        printf("%d ", stack1[i]);
    }
}

int main()
{
    int n, i, x;

    printf("Enter number of elements: " );
    scanf("%d", &n);

    printf("Enter elements: " );

    for (i = 0; i < n; i++)
    {
        scanf("%d", &x);
        enqueue(x);
    }

    dequeue();
    display();

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter elements: 10 20 30 40 50

Queue elements: 20 30 40 50
```

3) Write a C program using stack data structure to manage history of webpages visited.

Source Code:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

char history[MAX][50];
int top = -1;

void visit(char page[])
{
    strcpy(history[++top], page);
}

void back()
{
    if (top == -1)
    {
        printf("\nNo history available." );
    }
    else
    {
        printf("\nGoing back from: %s", history[top]);
        top--;
    }
}

void display()
{
    int i;

    printf("\n--- Webpage History ---\n" );

    for (i = top; i >= 0; i--)
    {
        printf("%s\n", history[i]);
    }
}

int main()
{
    visit("Google");
    visit("YouTube");
    visit("Wikipedia");
    visit("GitHub");

    display();
    back();
    display();

    return 0;
}
```

Output:

```
--- Webpage History ---
GitHub
Wikipedia
YouTube
Google

Going back from: GitHub
--- Webpage History ---
Wikipedia
YouTube
Google
```

4) Write a C program to sort a stack using another stack.

Source Code:

```
#include <stdio.h>

#define MAX 100

int stack[MAX], temp[MAX];
int top = -1, ttop = -1;

void push(int s[], int *top, int x)
{
    s[++(*top)] = x;
}

int pop(int s[], int *top)
{
    return s[(*top)--];
}

void sortStack()
{
    int x;

    while (top != -1)
    {
        x = pop(stack, &top);

        while (ttop != -1 && temp[ttop] > x)
        {
            push(stack, &top, pop(temp, &ttop));
        }

        push(temp, &ttop, x);
    }

    while (ttop != -1)
    {
        push(stack, &top, pop(temp, &ttop));
    }
}

void display()
{
    int i;

    printf("\nSorted stack: " );

    for (i = top; i >= 0; i--)
    {
        printf("%d ", stack[i]);
    }
}

int main()
{
    int n, i, x;

    printf("Enter number of elements: " );
    scanf("%d", &n);

    printf("Enter elements: " );

    for (i = 0; i < n; i++)
    {
        scanf("%d", &x);
        push(stack, &top, x);
    }

    sortStack();
    display();

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter elements: 30 10 50 20 40
```

```
Sorted stack: 10 20 30 40 50
```

5) The stock span problem is a financial problem where we have a series of daily price quotes for a stock denoted by an array and the task is to calculate the span of the stock's price for all days. The span of the stock price of the ith day gives the maximum number of consecutive days leading up to ith day (including current day) where stock price is less than or equal to its price on day i.

Source Code:

```
#include <stdio.h>

#define MAX 100

int main()
{
    int price[MAX], span[MAX], stack[MAX];
    int n, i, top = -1;

    printf("Enter number of days: " );
    scanf("%d", &n);

    printf("Enter stock prices: " );

    for (i = 0; i < n; i++)
    {
        scanf("%d", &price[i]);
    }

    for (i = 0; i < n; i++)
    {
        while (top != -1 && price[stack[top]] <= price[i])
        {
            top--;
        }

        if (top == -1)
        {
            span[i] = i + 1;
        }
        else
        {
            span[i] = i - stack[top];
        }

        stack[++top] = i;
    }

    printf("\nStock span: " );

    for (i = 0; i < n; i++)
    {
        printf("%d ", span[i]);
    }

    return 0;
}
```

Output:

```
Enter number of days: 7
Enter stock prices: 100 80 60 70 60 75 85

Stock span: 1 1 1 2 1 4 6
```

6) Given an array, for each element find the nearest element to its right that has a higher frequency than the current element. If no such element, set the value to -1. Implement using stack.

Source Code:

```
#include <stdio.h>

#define MAX 100

int main()
{
    int arr[MAX], freq[MAX] = {0}, stack[MAX], result[MAX];
    int n, i, j, top = -1;

    printf("Enter number of elements: " );
    scanf("%d", &n);

    printf("Enter elements: " );

    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (arr[i] == arr[j])
            {
                freq[i]++;
            }
        }
    }

    for (i = n - 1; i >= 0; i--)
    {
        while (top != -1 && freq[stack[top]] <= freq[i])
        {
            top--;
        }

        if (top == -1)
        {
            result[i] = -1;
        }
        else
        {
            result[i] = arr[stack[top]];
        }

        stack[++top] = i;
    }

    printf("\nResult: " );

    for (i = 0; i < n; i++)
    {
        printf("%d ", result[i]);
    }

    return 0;
}
```

Output:

```
Enter number of elements: 6
Enter elements: 2 1 1 3 2 1

Result: 1 -1 -1 2 1 -1
```

7) Given a pattern containing only I's and D's, I for increasing and D for decreasing. Design an algorithm using stack to print the minimum number following that pattern. Digits are from 1-9 and digits cannot repeat.

Source Code:

```
#include <stdio.h>
#include <string.h>

#define MAX 20

int main()
{
    char pattern[MAX];
    int stack[MAX], top = -1;
    int i, n, num = 1;

    printf("Enter pattern: " );
    scanf("%s", pattern);

    n = strlen(pattern);

    printf("\nMinimum number: " );

    for (i = 0; i <= n; i++)
    {
        stack[++top] = num++;

        if (i == n || pattern[i] == 'I')
        {
            while (top != -1)
            {
                printf("%d", stack[top--]);
            }
        }
    }

    return 0;
}
```

Output:

```
Enter pattern: DIDI

Minimum number: 21435
```

8) Given an array of digits (values from 0 to 9), find the minimum possible sum of two numbers formed from the digits of the array. All digits of the given array must be used to form the two numbers. Use queue for implementation.

Source Code:

```
#include <stdio.h>

#define MAX 100

int main()
{
    int digits[MAX], queue[MAX];
    int n, i, front = 0, rear = 0;
    int num1 = 0, num2 = 0;

    printf("Enter number of digits: " );
    scanf("%d", &n);

    printf("Enter digits: " );

    for (i = 0; i < n; i++)
    {
        scanf("%d", &digits[i]);
    }

    for (i = 0; i < n - 1; i++)
    {
        int j;

        for (j = i + 1; j < n; j++)
        {
            if (digits[i] > digits[j])
            {
                int temp = digits[i];
                digits[i] = digits[j];
                digits[j] = temp;
            }
        }
    }

    for (i = 0; i < n; i++)
    {
        queue[rear++] = digits[i];
    }

    while (front < rear)
    {
        num1 = num1 * 10 + queue[front++];

        if (front < rear)
        {
            num2 = num2 * 10 + queue[front++];
        }
    }

    printf("\nNumbers: %d and %d", num1, num2);
    printf("\nMinimum sum: %d", num1 + num2);

    return 0;
}
```

Output:

```
Enter number of digits: 6
Enter digits: 6 8 4 5 2 3

Numbers: 246 and 358
Minimum sum: 604
```

9) Given an array and a positive integer k, find the first negative integer for each window (contiguous subarray) of size k. If a window does not contain any negative integer print 0 for that window. Implement using queue.

Source Code:

```
#include <stdio.h>

#define MAX 100

int main()
{
    int arr[MAX], queue[MAX];
    int n, k, i, front = 0, rear = 0;

    printf("Enter number of elements: " );
    scanf("%d", &n);

    printf("Enter elements: " );

    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: " );
    scanf("%d", &k);

    printf("\nFirst negative integers: " );

    for (i = 0; i < n; i++)
    {
        if (arr[i] < 0)
        {
            queue[rear++] = i;
        }

        if (i >= k - 1)
        {
            while (front < rear && queue[front] < i - k + 1)
            {
                front++;
            }

            if (front < rear)
            {
                printf("%d ", arr[queue[front]]);
            }
            else
            {
                printf("0 " );
            }
        }
    }

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter elements: -8 2 3 -6 1
Enter k: 2

First negative integers: -8 0 -6 -6
```

10) Given a matrix m*n where each cell has the value 0, 1 or 2: 0 = empty cell, 1 = fresh orange, 2 = rotten orange. A rotten orange can rot its neighbours (left, right, up, down). Find the minimum time required so that all oranges become rotten. If impossible, return -1. Use queue for implementation.

Source Code:

```
#include <stdio.h>

#define MAX 20

struct cell
{
    int row;
    int col;
};

int main()
{
    int a[MAX][MAX], m, n;
    struct cell queue[MAX * MAX];
    int front = 0, rear = 0;
    int i, j, time = 0, fresh = 0;
    int r, c;

    printf("Enter rows and columns: " );
    scanf("%d %d", &m, &n);

    printf("Enter matrix:\n" );

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);

            if (a[i][j] == 2)
            {
                queue[rear].row = i;
                queue[rear].col = j;
                rear++;
            }
            else if (a[i][j] == 1)
            {
                fresh++;
            }
        }
    }

    while (front < rear && fresh > 0)
    {
        int count = rear - front;

        for (i = 0; i < count; i++)
        {
            r = queue[front].row;
            c = queue[front].col;
            front++;

            if (r > 0 && a[r - 1][c] == 1)
            {
                a[r - 1][c] = 2;
                fresh--;
                queue[rear].row = r - 1;
                queue[rear].col = c;
                rear++;
            }

            if (r < m - 1 && a[r + 1][c] == 1)
            {
                a[r + 1][c] = 2;
                fresh--;
                queue[rear].row = r + 1;
                queue[rear].col = c;
                rear++;
            }

            if (c > 0 && a[r][c - 1] == 1)
            {
                a[r][c - 1] = 2;
```

```
                fresh--;
                queue[rear].row = r;
                queue[rear].col = c - 1;
                rear++;
            }

            if (c < n - 1 && a[r][c + 1] == 1)
            {
                a[r][c + 1] = 2;
                fresh--;
                queue[rear].row = r;
                queue[rear].col = c + 1;
                rear++;
            }
        }

        time++;
    }

    if (fresh == 0)
    {
        printf("\nMinimum time: %d", time);
    }
    else
    {
        printf("\nMinimum time: -1" );
    }

    return 0;
}
```

Output:

```
Enter rows and columns: 3 3
Enter matrix:
2 1 1
1 1 0
0 1 1

Minimum time: 4
```